

```
#!/usr/bin/env python3
```

```
"""
```

```
Telegram-бот: MTS → MP4 конвертер
```

```
=====
```

Установка:

```
pip install python-telegram-bot ffmpeg-python
```

Требования:

ffmpeg должен быть установлен в системе:

- Ubuntu/Debian: `sudo apt install ffmpeg`

- Railway/Render: добавить в `nixpacks.toml` или `Dockerfile` (см. ниже)

Запуск:

```
BOT_TOKEN=ваш_токен python bot.py
```

Получить токен:

1. Написать @BotFather в Telegram
2. /newbot → следовать инструкциям
3. Скопировать токен

```
"""
```

```
import asyncio
```

```
import logging
```

```
import os
```

```
import subprocess
```

```
import tempfile
```

```
from pathlib import Path
```

```
from telegram import Update
```

```
from telegram.constants import ParseMode
```

```
from telegram.ext import (
```

```
    Application,
```

```
    CommandHandler,
```

```
    ContextTypes,
```

```
    MessageHandler,
```

```
    filters,
```

```
)
```

```
# — Настройки —————
```

```
BOT_TOKEN = os.environ.get("BOT_TOKEN", "ВСТАВЬТЕ_ТОКЕН_СЮДА")
```

```
MAX_FILE_SIZE_MB = 2000 # лимит Telegram для ботов — 2 ГБ (Bot API)
```

```
logging.basicConfig(
```

```

        format="%asctime)s | %(levelname)s | %(message)s",
        level=logging.INFO,
    )
log = logging.getLogger(__name__)

# ——— Утилиты —————

def check_ffmpeg() -> bool:
    try:
        subprocess.run(["ffmpeg", "-version"], capture_output=True, check=True)
        return True
    except (FileNotFoundError, subprocess.CalledProcessError):
        return False

def convert_mts_to_mp4(src: Path, dst: Path) -> tuple[bool, str]:
    """Конвертирует файл. Возвращает (успех, сообщение_об_ошибке)."""
    cmd = [
        "ffmpeg", "-y",
        "-i", str(src),
        "-c:v", "copy",    # видео без перекодирования — быстро!
        "-c:a", "aac",     # аудио → AAC
        "-b:a", "192k",
        str(dst),
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode != 0:
        return False, result.stderr[-500:]
    return True, ""

def human_size(path: Path) -> str:
    size = path.stat().st_size
    for unit in ("Б", "КБ", "МБ", "ГБ"):
        if size < 1024:
            return f"{size:.1f} {unit}"
        size /= 1024
    return f"{size:.1f} ТБ"

# ——— Хэндлеры —————

async def cmd_start(update: Update, ctx: ContextTypes.DEFAULT_TYPE) -> None:
    await update.message.reply_text(
        "👋 *MTS → MP4 Конвертер*\n\n"
        "Просто отправь мне файл *.MTS* — я верну его в формате *.MP4*.\n\n"

```

```

        "⚠️ Файлы до *2 ГБ* (ограничение Telegram).\n"
        "⚡ Видео копируется без перекодирования — быстро!",
        parse_mode=ParseMode.MARKDOWN,
    )

async def cmd_help(update: Update, ctx: ContextTypes.DEFAULT_TYPE) -> None:
    await update.message.reply_text(
        "📖 *Как пользоваться:*\n\n"
        "1. Отправь файл `.MTS` как документ\n"
        "2. Подожди немного\n"
        "3. Получи `.MP4`\n\n"
        "Команды:\n"
        "/start — приветствие\n"
        "/help — эта справка",
        parse_mode=ParseMode.MARKDOWN,
    )

async def handle_document(update: Update, ctx: ContextTypes.DEFAULT_TYPE) -> None:
    doc = update.message.document
    filename = doc.file_name or ""

    # Проверяем расширение
    if not filename.lower().endswith(".mts"):
        await update.message.reply_text(
            "⚠️ Это не .MTS файл.\nОтправь файл с расширением `.MTS`."
        )
        return

    size_mb = doc.file_size / 1024 / 1024
    status_msg = await update.message.reply_text(
        f"🕒 Получаю файл `{filename}` ({size_mb:.1f} МБ)..."
    )
    await status_msg.edit_text(f"⚡ Скачиваю файл `{filename}`", parse_mode=ParseMode.MARKDOWN)

    with tempfile.TemporaryDirectory() as tmp_dir:
        tmp = Path(tmp_dir)
        src = tmp / filename
        dst = tmp / (Path(filename).stem + ".mp4")

        # Скачиваем файл
        try:
            tg_file = await ctx.bot.get_file(doc.file_id)
            await tg_file.download_to_drive(str(src))
        except Exception as e:
            await status_msg.edit_text(f"❌ Ошибка скачивания:\n`{e}`", parse_mode=ParseMode.MARKDOWN)

```

```

        return

# Конвертируем
await status_msg.edit_text(
    f" Ошибка отправки файла: \n`{e}`",
        parse_mode=ParseMode.MARKDOWN,
    )

async def handle_other(update: Update, ctx: ContextTypes.DEFAULT_TYPE) -> None:
    await update.message.reply_text(
        "Отправь мне файл `.MTS` как документ (не как видео/фото).\n"
        "/help — справка"
    )

```

```
)

# — Запуск —————

def main() -> None:
    if BOT_TOKEN == "ВСТАВЬТЕ_ТОКЕН_СЮДА":
        log.error("Укажите BOT_TOKEN! Переменная окружения или впишите в код.")
        return

    if not check_ffmpeg():
        log.error(
            "ffmpeg не найден! Установите: apt install ffmpeg или brew install
        )
        return

    log.info("Бот запускается...")
    app = Application.builder().token(BOT_TOKEN).build()

    app.add_handler(CommandHandler("start", cmd_start))
    app.add_handler(CommandHandler("help", cmd_help))
    app.add_handler(MessageHandler(filters.Document.ALL, handle_document))
    app.add_handler(MessageHandler(filters.ALL, handle_other))

    log.info("Бот запущен. Ожидаю сообщения...")
    app.run_polling(drop_pending_updates=True)

if __name__ == "__main__":
    main()
```